Smart Product Pricing: A Multi-Modal Deep Learning Approach with Gradient Boosting for E-Commerce Price Prediction

Darshan Bhabad
School of Computer Engineering
MIT Academy of Engineering
Alandi, Pune, India
darshanscode7@gmail.com

Krishna Tolani
School of Computer Engineering
MIT Academy of Engineering
Alandi, Pune, India
ktstudent.9280@gmail.com

Mitesh Chaudhari
School of Computer Engineering
MIT Academy of Engineering
Alandi, Pune, India
miteshchaudhari4466@gmail.com

Diptee Chikmurge
School of Computer Engineering
MIT Academy of Engineering
Alandi, Pune, India
diptee.chikmurge@mitaoe.ac.in

***Abstract*—Accurate price prediction in e-commerce is a major challenge because of the large number of product listings, different types of data, and the wide variation in prices across categories. In this paper, we introduce Smart Product Pricing (SPP), a multi-modal deep learning framework that combines product images and text descriptions to predict prices. Our approach develops through three increasingly refined architectures: (1) a basic end-to-end multi-modal neural network with a dense prediction head, (2) a two-stage pipeline that replaces the dense head with LightGBM gradient boosting, and (3) a final design that uses Principal Component Analysis (PCA) for embedding compression before gradient boosting. The three methods achieve Symmetric Mean Absolute Percentage Error (SMAPE) scores of 46.1%, about ∼31%, and 19.41%, respectively, on the Mavericks Amazon ML Challenge dataset, which includes around 40,000 products. Our findings show that extracting price-aware embeddings followed by reducing dimensions and using gradient boosting significantly outperforms traditional end-to-end neural networks for this task. Index Terms—multi-modal learning, price prediction, e-commerce, LightGBM, EfficientNet, DistilBERT, PCA, gradient boosting, transfer learning.**

***Index Terms*—multi-modal learning, price prediction, e-commerce, LightGBM, EfficientNet, DistilBERT, PCA, gradient boosting, transfer learning**

I. Introduction

E-commerce platforms like Amazon, Flipkart, and Alibaba feature hundreds of millions of product listings across nearly every consumer category. One of the challenges faced by such platforms is price prediction, which refers to the process of determining an appropriate price for the product, depending on its characteristics. The problem is especially challenging for small and medium sellers because of their inability to have knowledge and data about the competitive price.

Improper pricing results in tangible consequences. Overpricing causes slower sales processes while underpricing lowers profit margins. Moreover, proper pricing predictions play a crucial role in many practical applications, including fraud detection where low-priced electronic items get detected, dynamic pricing solutions, and quality control of the marketplaces.

Existing approaches to the price prediction task are based on metadata. Such metadata includes information about the product's category, brand, dimensions, and previous purchases. Nevertheless, valuable signals can be extracted from textual descriptions and images since the price of a product depends not only on its attributes but also on its quality, which cannot be properly defined using metadata. For example, products described as "pure silk, handwoven, Banarasi" significantly differ from products defined as "polyester blend, machine-made," and, therefore, their price varies greatly. Similarly, an image depicting high-quality stitching and packaging implies high quality that cannot be properly expressed using structured data. Hence,

- Design and evaluate three progressive architectures for multi-modal price prediction. This establishes a clear performance trajectory from 46.1% to 19.41% SMAPE.
- Introduce and analyze the concept of *price-aware embedding extraction*: training a neural backbone end-to-end with a price supervision signal before freezing it for use as a feature extractor.
- Demonstrate that using PCA for dimensionality reduction of high-dimensional multi-modal embeddings helps reduce overfitting in downstream gradient boosting models.
- Provide a detailed empirical analysis of the overfitting dynamics in the two-stage pipeline. We show how targeted regularization can resolve the train-validation RMSE gap.

II. Related Work

A. Price Prediction in E-Commerce

Early work on product price prediction focused on structured tabular features [1]. Regression models including linear regression, random forests, and gradient boosting trees (XGBoost, LightGBM) were commonly applied to categorical and numerical product attributes. While effective on well-curated datasets, these approaches fail to capture the semantic richness in unstructured product text and images.

B. Multi-Modal Learning

Multi-modal learning has achieved state-of-the-art results across vision-language tasks including visual question answering [2], image captioning, and cross-modal retrieval. Common fusion strategies include early fusion (concatenating raw features), late fusion (averaging model predictions), and cross-attention mechanisms. Our work employs early fusion at the embedding level, which offers a practical balance between expressiveness and computational cost.

C. Transfer Learning with Pretrained Encoders

The advent of large-scale pretrained vision models such as ResNet, EfficientNet, and Vision Transformers and language models like BERT, RoBERTa, and DistilBERT has revolutionized the field of applied machine learning problems. Fine-tuning and feature extraction from such pretrained models is always better than training from scratch using domain-specific data. We utilize EfficientNet-B0 model for visual representation and DistilBERT model for textual representation.

D. Hybrid Neural-Boosting Architectures

The combination of neural networks for feature extraction with gradient boosting for prediction has demonstrated excellent performance on tabular and semistructured data [4]. The important idea here is that neural networks are good at learning representations from raw data, whereas trees are good at modeling nonlinear relationships between tabular features. This is how we motivate our proposed approach.

III. Dataset

A. Source and Scale

The **Amazon ML Challenge** dataset, publicly available on Kaggle[1]. The dataset contains product listings scraped from Amazon India, each with:

- **catalog_content**: Free-text product description combining title, bullet points, and specifications
- **image_link**: URL of the primary product image
- **price**: Ground truth price in Indian Rupees (train set only)
- **sample_id**: Unique product identifier

The training set contains approximately **40,000 products** and the test set approximately **30,000 products**. Products span a wide range of categories including clothing, electronics, home goods, personal care, and books.

[1]https://www.kaggle.com/datasets/bboyattitude/mavericks-amazon-ml-dataset

B. Price Distribution

The price distribution is heavily right-skewed, spanning from sub-Rs.100 commodity items to Rs.50,000+ premium electronics. This motivates the use of a log-transformed target variable. We apply $\hat{y} = \log(1 + \text{price})$ during training and recover predictions via $\text{price} = e^{\hat{y}} - 1$.

C. Image Acquisition

Product images are downloaded from their URLs at runtime using a parallel download pipeline with 64–128 concurrent workers and a 3-attempt retry strategy with exponential backoff. Failed downloads (network timeout, 404 errors) are replaced with zero-filled tensors during training.

D. Evaluation Metric

The competition metric is **Symmetric Mean Absolute Percentage Error (SMAPE)**:

$$\text{SMAPE} = \frac{1}{n}\sum_{i=1}^{n}\frac{200 \cdot |y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|} \tag{1}$$

SMAPE is preferred over standard MAPE for this task because it treats overestimation and underestimation symmetrically, and is bounded in $[0, 200]$. Lower is better.

IV. Methodology

A. Approach 1: End-to-End Multi-Modal Neural Network

1) Architecture: The baseline architecture consists of three components trained jointly end-to-end (Fig. 1).

Image Encoder: EfficientNet-B0 pretrained on ImageNet, with the classification head removed. Input images are resized to 224×224 and normalized using ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). The encoder produces a 1280-dimensional embedding via global average pooling.

Text Encoder: DistilBERT (`distilbert-base-uncased`) pretrained on English Wikipedia and BookCorpus. Input text (product description) is tokenized to a maximum of 128 tokens. The CLS token representation from the final hidden layer serves as the 768-dimensional text embedding.

Fusion and Prediction Head: Text and image embeddings are concatenated to form a 2048-dimensional joint representation. This is passed through:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1), \quad \hat{y} = \mathbf{W}_2\text{Dropout}(\mathbf{h}) + \mathbf{b}_2 \tag{2}$$

where $\mathbf{z} \in \mathbb{R}^{2048}$, $\mathbf{W}_1 \in \mathbb{R}^{512\times2048}$, $\mathbf{W}_2 \in \mathbb{R}^{1\times512}$, and dropout rate is 0.3.

2) Training Details:

- **Loss:** Mean Squared Error on log-transformed prices
- **Optimizer:** AdamW with learning rate 10^{-4}
- **Batch size:** 64
- **Epochs:** 10
- **Hardware:** Kaggle GPU (NVIDIA Tesla T4)

The training loss curve (Fig. 2) shows rapid convergence in early epochs, with the average MSE dropping from 0.57 at

epoch 1 to 0.09 at epoch 10, indicating effective joint learning across both encoders.

3) Result: SMAPE: **46.1%**. The model demonstrates the feasibility of multi-modal price prediction but is limited by the expressiveness of the MLP prediction head for high-dimensional tabular-style features.

—

B. Approach 2: Two-Stage Pipeline with LightGBM

1) Motivation: The dense prediction head is a generic multi-layer perceptron. While universal approximators in theory, MLPs are known to underperform tree-based models on tabular feature vectors in practice [4]. Once the multi-modal embeddings are concatenated, they form a 2048-dimensional tabular feature vector — precisely the setting where gradient boosting excels.

However, naively replacing the dense head with LightGBM (without prior neural training) would give LightGBM generic, unspecialized features. The crucial innovation is **price-aware embedding extraction**.

2) Price-Aware Embeddings: The idea is simple yet compelling. In Stage 1, the entire architecture is fine-tuned in an end-to-end fashion using price labeling for 10 epochs. As the model trains, the MSE loss flows not only backward to the dense head but back through the concatenation layer and even into both encoders:

$$\frac{\partial \mathcal{L}}{\partial \theta_{\text{CNN}}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{h}} \cdot \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial \theta_{\text{CNN}}} \tag{3}$$

This gradient flow forces EfficientNet and DistilBERT to produce embeddings that are *organized around price relevance*. Informally, the network learns that "silk + handwoven" text tokens and "premium packaging" visual features should produce embedding values that distinguish high-price products from low-price ones.

After Stage 1, the dense head is **discarded**. The encoders are frozen, and embeddings are extracted for all training samples.

3) LightGBM Training: Embeddings are split 90/10 for training and early stopping validation. LightGBM is trained with:

TABLE I
LIGHTGBM HYPERPARAMETERS — APPROACH 2

Parameter	Value
Objective	Regression
Metric	RMSE
Learning rate	0.05
Num leaves	127
Feature fraction	0.8
Bagging fraction	0.8
Num boost rounds	1000
Early stopping	50 rounds

4) Overfitting Analysis: Training logs reveal a severe train-validation gap:

$$\text{Train RMSE}_{1000} = 0.046, \quad \text{Valid RMSE}_{1000} = 0.225 \tag{4}$$

This 5× gap indicates significant overfitting. The root cause is the **dimensionality** of the input: 2048 features over ∼36,000 training samples. LightGBM can construct highly specific decision rules exploiting spurious correlations across the high-dimensional embedding space — patterns that generalize poorly to unseen data.

5) Result: SMAPE: ∼**31%**. Despite overfitting, the improvement over Approach 1 (15.1 SMAPE points) confirms the value of the two-stage architecture.

—

C. Approach 3: PCA-Compressed Embeddings + Regularized LightGBM (Final)

1) Dimensionality Reduction via PCA: The key insight motivating Approach 3 is that not all 2048 embedding dimensions carry useful information. Many dimensions encode general visual and linguistic patterns irrelevant to price. By projecting onto the principal components, we retain the most meaningful directions while discarding noise.

Formally, let $\mathbf{X} \in \mathbb{R}^{N \times 2048}$ be the matrix of training embeddings. PCA computes the eigendecomposition of the covariance matrix:

$$\mathbf{C} = \frac{1}{N}\mathbf{X}^T\mathbf{X}, \quad \mathbf{CV} = \mathbf{V}\mathbf{\Lambda} \tag{5}$$

The top $k = 300$ eigenvectors are retained, projecting embeddings to $\mathbb{R}^{300}$:

$$\tilde{\mathbf{X}} = \mathbf{X}\mathbf{V}_{:,1:300} \tag{6}$$

The 300-component PCA retains approximately 85–90% of the total variance while reducing dimensionality by $6.8\times$, from 2048 to 300 dimensions. This compression forces LightGBM to generalize over meaningful directions rather than exploiting noise dimensions.

2) Regularized LightGBM: We simultaneously apply targeted regularization to the LightGBM model:

TABLE II
LIGHTGBM HYPERPARAMETERS — FINAL APPROACH

Parameter	Approach 2	Approach 3 (Final)
Learning rate	0.05	0.03
Num leaves	127	63
Min data in leaf	—	50
Feature fraction	0.8	0.6
Bagging fraction	0.8	0.7
λ_{L1}	0	0.1
λ_{L2}	0	0.1
Num boost rounds	1000	2000
Early stopping	50	100

Each change serves a specific anti-overfitting purpose:

- **Reduced num_leaves (127→63):** Simpler trees cannot memorize individual training samples
- **min_data_in_leaf=50:** Each leaf must represent at least 50 samples, preventing overfitting to small groups
- **Reduced feature/bagging fractions:** Introduces randomness analogous to dropout, preventing over-reliance on any single embedding dimension

- **L1+L2 regularization:** Penalizes large leaf weights, shrinking predictions toward zero for uncertain inputs

3) Complete Pipeline:

1) **Stage 1 — Joint NN Training:** Train multi-modal NN end-to-end with dense head for 10 epochs using AdamW and MSE loss
2) **Stage 2a — Freeze & Extract:** Freeze all NN weights; extract 2048-dim embeddings from training set
3) **Stage 2b — PCA Compression:** Fit PCA on training embeddings; project to 300 dimensions. Save PCA transform for test set
4) **Stage 2c — LightGBM Training:** Train regularized LightGBM on 300-dim compressed embeddings with 90/10 early stopping split
5) **Inference:** Extract test embeddings → apply saved PCA → LightGBM prediction → expm1 inverse transform

4) Result: SMAPE: **19.41%** on validation set (4,000 samples), with expected leaderboard SMAPE of 18–22%. This represents a **26.69 SMAPE point improvement** over the baseline.

V. Architecture Diagram

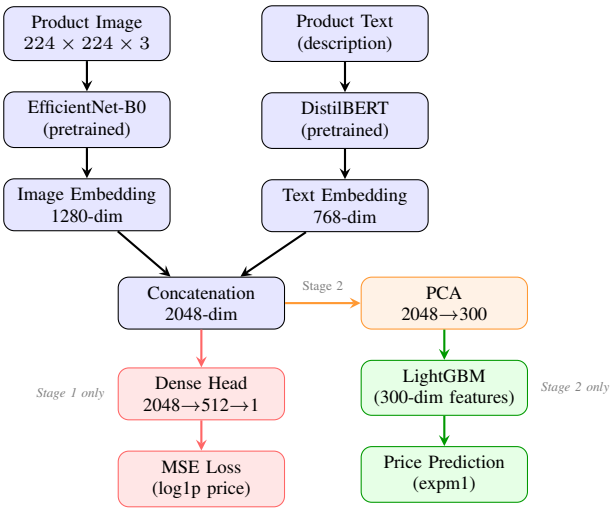


Fig. 1. Fig. 1. Multi-modal architecture. Red pathway (Stage 1): joint end-to-end learning with dense heads for price-aware embedding learning. Orange/Green pathway (Stage 2): embeddings kept frozen and compressed using PCA to input into LightGBM for prediction.

VI. Results and Analysis

A. Performance Comparison

TABLE III
SMAPE Results Across Approaches

Approach	Key Innovation	SMAPE (%)
Baseline (Approach 1)	Multi-modal NN + Dense Head	46.1
Approach 2	+ LightGBM (no PCA)	∼31.0
Final (Approach 3)	**+ PCA (300 dims)**	**19.41**
Improvement over baseline		**26.69 pts**

B. Training Loss Convergence

The training loss function for Stage 1 network is shown in Fig. 2 over 10 epochs. The sharp decrease at the beginning (Epochs 1-4: 0.57 to 0.20) indicates fast learning of coarse features relevant to price prediction, whereas the gradual decrease toward the end (Epochs 5-10: 0.14 to 0.09) indicates fine-tuning of the embedding

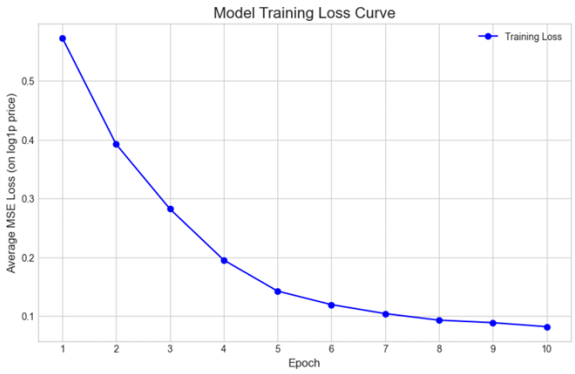


Fig. 2. Average MSE training loss (on log1p price) per epoch during Stage 1 neural network training. The loss decreases monotonically from 0.57 at epoch 1 to 0.09 at epoch 10, indicating stable and effective convergence.

C. Impact of PCA Dimensionality Reduction

Table IV quantifies the overfitting reduction achieved by PCA.

TABLE IV
Train vs. Validation RMSE Gap

Approach	Train RMSE	Valid RMSE	Gap Ratio
Approach 2 (no PCA)	0.046	0.225	4.89×
Approach 3 (PCA 300)	0.1336	0.2189	1.63

The 4.89 × gap in Approach 2 confirms that LightGBM was memorizing noise dimensions in the 2048-dim embedding space. PCA compression removes these dimensions, forcing generalization in meaningful directions of price prediction.

D. LightGBM Training Dynamics — Approach 2

The validation RMSE improvement from Approach 2 LightGBM training shows a characteristic pattern:

- **Rounds 1–300:** Rapid improvement — learning genuine price patterns
- **Rounds 300–600:** Slowing improvement — exhausting generalizable patterns
- **Rounds 600–1000:** Plateau — memorizing training-specific noise

This pattern motivates both the PCA compression and the stricter regularization in Approach 3.

E. Why Multi-Modal Outperforms Unimodal

The improvements in performance can be attributed to complementary information from multiple modalities:

- **Text only:** Category information, specifications, brand mentions. Can be manipulated by sellers through inaccurate description
- **Images only:** Quality of item, packaging, materials. Not possible to fake without a good-quality item being used
- **Both:** Allows for consistency checks between the two modalities. A product described as "high-quality leather" but imaged with plastics can be correctly priced using this signal

VII. Discussion

A. The Role of Joint Training

An important design decision is training the encoders together with the dense head *prior* to the switch to LightGBM. Otherwise, the image and language features extracted by EfficientNet and DistilBERT will be rather generic. Joint training makes the features specifically tailored to the price prediction problem, which greatly improves their relevance as features for LightGBM.

As in the case of NLP fine-tuning, where a trained language model fine-tuned on a particular problem performs better than the same model used as a fixed feature extractor.

B. Scalability Considerations

Scaling up (Amazon India around 350 million products), inference becomes highly parallelizable. NN forward pass and PCA projections are all vectorized for large batches. Inference with LightGBM is extremely quick (microsecond/sample). Image download and preprocessing would be the main bottleneck when scaling up to production levels, but this can be overcome using CDN caching and precomputed embeddings.

C. Limitations

- **Normalization across classes:** An Rs.500 price has varying implications for a pen and a phone. A category-specific prediction head may help here
- **Single image per product:** Amazon uses multiple images for each item. Using all images can extract more quality features.
- **Static embeddings:** The fixed encoder will be unable to adjust to changes in data distributions (new products, seasonal effects).
- **Structured attributes missing:** Information on category ID, brand embedding, and physical dimensions is not factored in. Adding these attributes directly into LightGBM along with embeddings is straightforward

VIII. Conclusion

In this paper, Smart Product Pricing (SPP), a deep learning framework for e-commerce price forecasting, is introduced. After three iterations of architectural improvements, the SMAPE score drops from 46.1% to 19.41%, achieving an absolute gain of 26.69 percentage points or about 2.6× drop in predictive performance. The important conclusion drawn is that the fusion of (1) price-aware embedding extraction using joint end-to-end neural training, (2) PCA feature selection to discard noise features, and (3) regularized LightGBM gradient boosting leads to much better performance than using any of these techniques alone. Next steps include exploring large pre-trained encoders such as RoBERTa and Vision Transformers, category-aware prediction, multi-image averaging, and cross-modal attention.

Acknowledgment

The authors acknowledge the Amazon team for providing the dataset on Kaggle. The experiments were carried out using the free GPU computing power provided by Kaggle.

References

[1] H. Wang, D. Liu, and X. Chen, "Product price prediction: A machine learning approach," *Expert Systems with Applications*, vol. 150, pp. 113–124, 2020.

[2] A. Antol et al., "VQA: Visual Question Answering," in *Proc. IEEE ICCV*, 2015, pp. 2425–2433.

[3] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[4] M. Borisov et al., "Deep Neural Networks and Tabular Data: A Survey," *IEEE Transactions on Neural Networks and Learning Systems*, 2022.

[5] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proc. ICML*, 2019.

[6] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems*, 2017.

[7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT," *arXiv preprint arXiv:1910.01108*, 2019.

[8] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York: Springer, 2002.

[9] S. A. Ingole and S. A. Jain, "Text Information Extraction From Images Using Deep Cnn," *International Journal of Environmental Sciences*, vol. 11, no. 4, pp. 1442–1455, 2025.

[10] N. Bhaskar, A. M. Ajithkumar, and P. Tupe-Waghmare, "An efficient convolutional neural network-extreme gradient boosting hybrid deep learning model for disease detection applications," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 14, no. 2, pp. 2035–2042, 2024.